ADAMAS UNIVERSITY

KOLKATA

DATABASE MANAGEMENT SYSTEM

CONCISE ACADEMIC ASSIGNMENT

CSE303 | Database Management System

PREPARED BY	SUBMITTED TO
Titli Sadhukhan	Abhinandan Ghosh
PROGRAMME	COURSE
BCA (Hons.)	CSE303 – DBMS
ACADEMIC SESSION	SUBMISSION DATE
Semester V | Year 3rd	06 November 2026
ROLL NUMBER	REGISTRATION NUMBER
UG/SOET/29/24/076	AU/2024/0001212

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ABOUT THIS ASSIGNMENT

This concise assignment explains the principal concepts listed for CSE303. Each section combines a definition, key points, a practical example, and a short code illustration where useful.

The wording “dependency presentations” is standardised as dependency preservation. The final topic “data working vs data making” is interpreted as data warehousing vs. data mining, consistent with the supplied topic reference. Insertion and deletion are discussed as SQL operations and as database-design anomalies.

CONTENTS

1. Relational Calculus – Tuple and Domain Forms
2. Dependency Preservation
3. Query Optimisation
4. Database Security
5. DAC, MAC and RBAC
6. Insertion and Deletion
7. SQL Injection
8. Data Warehousing and Data Mining
9. Database Intrusion Detection
10. Conclusion and References

1. RELATIONAL CALCULUS

Relational calculus is a declarative query approach based on predicate logic. It describes the data a user wants, rather than the steps used to retrieve it.

Tuple Relational Calculus (TRC)

Definition: TRC uses a tuple variable to represent a complete row. Its general form is **{ t | P(t) }**, meaning all tuples t for which condition P is true.

Key points: tuple variables range over rows; predicates use comparisons and logical operators; existential (∃) and universal (∀) quantifiers may be used; safe expressions return finite results.

Example: list student rows for the BCA programme. Assume Student(student_id, student_name, programme).

```
{ s | Student(s) AND s.programme = 'BCA' }
```

Equivalent SQL:

```
SELECT *
FROM Student
WHERE programme = 'BCA';
```

```
EXPECTED OUTPUT / EFFECT
Returns all student records whose programme is BCA.
```

Domain Relational Calculus (DRC)

Definition: DRC uses variables for individual attribute values, not whole rows. Its general form is **{ <x1, x2, …, xn> | P(x1, x2, …, xn) }**.

Key points: each variable represents a column value; the query states which values to return; logical conditions and quantifiers define the required records.

Example: retrieve names of BCA students.

```
{ <name> | EXISTS id, prog (Student(id, name, prog) AND prog = 'BCA') }
```

```
SELECT student_name
FROM Student
WHERE programme = 'BCA';
```

```
EXPECTED OUTPUT / EFFECT
Returns only the student name column for matching students.
```

Difference: TRC works with tuple variables (rows); DRC works with domain variables (individual values). Both are non-procedural and, when restricted to safe expressions, have equivalent expressive power.

2. DEPENDENCY PRESERVATION

Dependency preservation is a property of a relational decomposition that helps keep functional rules enforceable after a table is split.

Definition: A decomposition is dependency-preserving when the functional dependencies of the original relation can be enforced by checking the decomposed relations separately, without joining them together.

Key points:

- Makes integrity checking simpler and often less costly.
- Helps maintain consistency after normalisation.
- Lossless join and dependency preservation are separate tests; a good decomposition should aim for both.
- A decomposition can be lossless but still fail to preserve a dependency.

Example: in a college, student_id determines student_name and dept_id; dept_id determines dept_name. Store department details separately to avoid repeating them for every student.

```
CREATE TABLE Department (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(70) NOT NULL,
  dept_id INT REFERENCES Department(dept_id)
);
```

The Student primary key enforces student_id → student_name and dept_id. The Department primary key enforces dept_id → dept_name. The rules can be checked locally in each table.

3. QUERY OPTIMISATION

Query optimisation selects an efficient execution plan for an SQL statement, aiming to reduce response time and resource use.

Definition: The DBMS compares alternative ways to execute a query and estimates their costs using table statistics, indexes, expected row counts, and available resources.

Main techniques

- **Rule-based rewriting:** simplify conditions, push filters closer to tables, and remove unused columns.
- **Cost-based optimisation:** estimate CPU, disk I/O, memory, and row counts before choosing a plan.
- **Join-order selection:** choose an efficient order for combining multiple tables.
- **Join algorithms:** nested-loop, hash, and sort-merge joins suit different data sizes and ordering.
- **Index use:** indexes can speed up searches, but excessive indexes increase storage and update costs.

Real-world example: an online store frequently looks up a customer's orders by customer number and date. A composite index can make that search faster.

```
CREATE INDEX ix_order_customer_date
ON Orders(customer_id, order_date);

SELECT order_id, order_date
FROM Orders
WHERE customer_id = 42
  AND order_date >= DATE '2026-01-01';
```

Illustrative plan: an index range scan examines the matching portion of the index rather than reading every order row. Confirm the actual plan with EXPLAIN (or the DBMS equivalent).

Good practice: retrieve only needed columns, index frequently filtered or joined columns, keep statistics current, and measure performance using realistic data.

4. DATABASE SECURITY

Database security protects data and database services against unauthorised viewing, alteration, loss, and disruption.

Definition: it is the combination of policies and technical controls used to protect database accounts, stored information, and database operations.

Three central objectives

- **Confidentiality:** information is visible only to authorised users.
- **Integrity:** information remains correct and is changed only in permitted ways.
- **Availability:** authorised users can access systems and data when required.

Important controls

- Authentication through strong passwords and multi-factor authentication.
- Authorisation through roles, privileges, and least-privilege access.
- Encryption for stored data and network connections.
- Audit logs to record access, changes, and administrative actions.
- Regular backups, tested recovery, updates, and secure configuration.

Real-world example: a hospital restricts doctors to clinical records, billing staff to payment information, and database administrators to system management. Logs and encryption protect patient information.

```
GRANT SELECT ON PatientRecords TO doctor_role;
REVOKE SELECT ON PatientRecords FROM doctor_role;
```

```
EXPECTED OUTPUT / EFFECT
The first statement grants read access; the second removes it. Exact privilege syntax varies by DBMS.
```

Security should be layered: no single control is sufficient on its own.

5. DAC, MAC AND RBAC

These access-control models define how a database decides which users may perform particular actions.

Discretionary Access Control (DAC)

The owner of an object can grant or revoke permissions. It is flexible and supports sharing, but permissions need careful management.

Example: a lecturer grants a teaching assistant read-only access to course marks.

```
GRANT SELECT ON CourseMarks TO assistant;
```

Mandatory Access Control (MAC)

A central policy controls access using security labels, such as Public, Confidential, or Secret, and user clearance levels. It is strict and suitable for highly sensitive environments, but less flexible.

Example: a user with Confidential clearance cannot open a Secret-level file.

Role-Based Access Control (RBAC)

Permissions are assigned to job roles, and users receive access through their assigned roles. It is practical for organisations with many users.

Example: a university may define Student, Lecturer, Accounts Officer, and Administrator roles.

```
CREATE ROLE lecturer;
GRANT SELECT ON Student TO lecturer;
GRANT lecturer TO user_titli;
```

Quick comparison

Model	Access is based on	Main benefit
DAC	Object-owner permissions	Flexible sharing
MAC	Security labels and central rules	Strict control
RBAC	Organisational roles	Easy administration

Organisations can combine models—for example, RBAC for daily job duties and classification rules for especially sensitive records.

6. INSERTION AND DELETION

INSERT and DELETE change table contents. The same terms also describe two common anomalies caused by poorly structured tables.

INSERT

INSERT adds new rows. The values must satisfy column types and constraints such as PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY.

```
INSERT INTO Student(student_id, student_name, programme)
VALUES (301, 'Titli', 'BCA');
```

```
EXPECTED OUTPUT / EFFECT
One student row is added if all constraints are satisfied.
```

DELETE

DELETE removes rows matching a condition. Omitting WHERE removes every row, so the target should always be checked.

```
DELETE FROM Student
WHERE student_id = 301;
```

```
EXPECTED OUTPUT / EFFECT
The row with student_id 301 is removed, if present.
```

Insertion anomaly: a new department cannot be stored in a combined Student table until a student is assigned to it.

Deletion anomaly: deleting the only student in a department may accidentally remove the department's only stored details.

Solution: normalise the design by keeping Department and Student in separate related tables. This allows department information to exist independently of student records.

Remember: DELETE removes selected rows, TRUNCATE removes all rows (behaviour varies by DBMS), and DROP removes the table structure.

7. SQL INJECTION

SQL injection is a vulnerability in which untrusted input changes the meaning of a database query.

Definition: it occurs when an application combines user input with SQL text in an unsafe way, causing input to be interpreted as part of the command.

Possible impact: attackers may read confidential records, bypass login checks, modify information, or delete data.

Unsafe pattern – illustration only

```
sql = "SELECT * FROM Users WHERE name = '"
      + user_input + "'";
```

The input is inserted directly into the SQL command. This can allow the command's logic to be altered.

Prevention

- Use prepared statements and parameterised queries.
- Validate input format and length; do not rely on validation alone.
- Give application accounts only the permissions they need.
- Avoid exposing detailed database errors to users.
- Review code and test applications for injection flaws.

Safer Python DB-API example

```
cursor.execute(
    "SELECT user_id FROM Users WHERE username = ?",
    (username_input,)
)
```

```
EXPECTED OUTPUT / EFFECT
The placeholder keeps the supplied username as a data value. Placeholder style depends on the database driver.
```

Real-world example: a shopping site searches for an order using a customer-entered order number. Parameter binding prevents that value from becoming SQL syntax.

8. DATA WAREHOUSING AND DATA MINING

A data warehouse organises historical information for analysis, while data mining discovers patterns and useful knowledge from datasets.

Data Warehousing

Definition: a data warehouse is a central repository that combines data from different sources for reporting, trend analysis, and decision support.

Key features: subject-oriented, integrated, time-variant, and generally non-volatile. Data is commonly prepared through ETL/ELT—extract, transform, and load.

Example: a retail company combines branch sales and online orders to compare monthly revenue.

```
SELECT product_id, SUM(quantity) AS units_sold
FROM Sales
GROUP BY product_id
ORDER BY units_sold DESC;
```

```
EXPECTED OUTPUT / EFFECT
Shows products ranked by total units sold.
```

Data Mining

Definition: data mining applies statistical and machine-learning methods to find patterns, relationships, groups, and unusual behaviour in data.

Common methods: classification predicts categories; clustering groups similar records; association rules find items that occur together; regression predicts numerical values; anomaly detection identifies unusual cases.

Example: a supermarket discovers that customers who buy tea often buy biscuits, and uses the pattern to plan promotions.

Difference at a glance

Aspect	Data warehouse	Data mining
Purpose	Store integrated history	Discover patterns
Output	Reports and dashboards	Rules, groups, predictions
Typical methods	ETL/ELT, OLAP	Statistics, machine learning

The warehouse provides organised data; mining analyses that data to discover insights. They are complementary, not interchangeable.

9. DATABASE INTRUSION DETECTION

Intrusion detection monitors database activity to identify suspicious access, misuse, and possible attacks.

Definition: a database intrusion detection system analyses database events, queries, accounts, and logs to detect behaviour that may violate security policy.

Approaches

- **Signature-based:** searches for known malicious patterns; may miss new attacks.
- **Anomaly-based:** flags activity that differs from normal behaviour.
- **Host-based:** monitors server logs and processes.
- **Network-based:** inspects database traffic.

Important functions: log activity, detect unusual query volumes, alert administrators, preserve evidence, and support incident response.

Example: an employee account that normally views a few customer records suddenly exports thousands of records late at night. Monitoring can flag the unusual activity.

Illustrative audit analysis

```
SELECT db_user, COUNT(*) AS read_count
FROM AuditEvents
WHERE object_name = 'Customer'
  AND action_name = 'SELECT'
GROUP BY db_user
HAVING COUNT(*) > 500;
```

```
EXPECTED OUTPUT / EFFECT
Illustratively flags accounts with more than 500 recorded reads. Audit table and column names depend on the DBMS.
```

Limitation: alerts can include false positives, and detection does not automatically stop an attack. Monitoring must be paired with investigation and response procedures.

10. CONCLUSION

Database management combines sound data design, efficient query processing, controlled access, and reliable analysis.

Relational calculus expresses queries declaratively, while dependency preservation helps maintain rules after decomposition. Query optimisation improves execution efficiency. Database security and access-control models protect information, and parameterised SQL helps prevent injection. Data warehouses support historical analysis, data mining discovers patterns, and intrusion detection helps identify suspicious activity.

Together, these concepts support databases that are consistent, efficient, secure, and useful for organisational decision-making.

REFERENCES

1. Silberschatz, Korth & Sudarshan, *Database System Concepts*, McGraw-Hill.
2. Elmasri & Navathe, *Fundamentals of Database Systems*, Pearson.
3. Ramakrishnan & Gehrke, *Database Management Systems*, McGraw-Hill.
4. PostgreSQL Documentation: https://www.postgresql.org/docs/
5. MySQL Reference Manual: https://dev.mysql.com/doc/refman/
6. OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/

— END —